

Exploring Regret Minimization and Its Variants in Normal-Form Games

Rozálie Bílková

Supervisor: doc. Ing. Tomáš Kroupa, Ph.D.

June 17, 2026



Strategic decision making

- ▶ Game theory studies interactions between multiple decision makers
- ▶ A player's outcome depends not only on their own strategy, but also on the strategies of others
- ▶ We focus on two-player zero-sum games

	R	P	S
R	0	-1	1
P	1	0	-1
S	-1	1	0

Rock-Paper-Scissors matrix

Finding optimal strategies

- ▶ Pure strategy: choose a single action
- ▶ Mixed strategy: probability over actions
- ▶ Nash equilibrium / minimax

Nash equilibrium

A pair of strategies where neither player can improve by unilaterally changing their strategy.

- ▶ In two player games: Nash equilibrium = minimax solution

Thesis objective

Goal

Evaluate whether Greedy Regret Minimization can improve convergence towards Nash equilibrium in two-player zero-sum games.

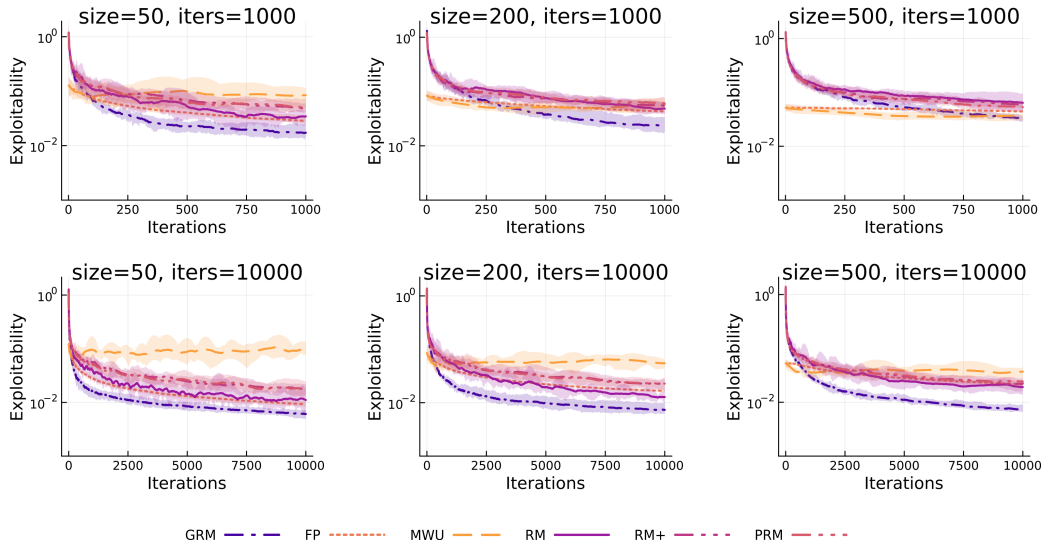
Implemented

- ▶ GRM algorithm
- ▶ regret minimization baselines
- ▶ evaluation framework

Evaluated

- ▶ multiple game classes
- ▶ exploitability convergence
- ▶ comparison with Double Oracle

Experimental Comparison Across Game Sizes



Regret

Main idea

Regret measures how much better we could have performed having always played the best fixed strategy in hindsight.

$$R = \underbrace{\text{our total loss}}_{\text{what happened}} - \underbrace{\text{best fixed action loss}}_{\text{hindsight}}$$

- ▶ Positive regret $\rightarrow$ we missed better strategies
- ▶ Negative regret $\rightarrow$ action performed poorly

From regret to equilibrium

Repeated game:

play strategy $\rightarrow$ observe loss $\rightarrow$ update regrets $\rightarrow$ new strategy

No-regret learning

If both player minimize regret in a zero-sum game, their average strategies converge to Nash equilibrium.

Example: **Regret Matching**

$$R^t = \sum_{\tau=1}^t r^{\tau}, \quad \sigma_i^{t+1} = \frac{[R_i^t]^+}{\sum_j [R_j^t]^+}$$

Greedy regret matching

Standard regret matching:

$$R_t = r_1 + r_2 + \cdots + r_t$$

Each iteration contributes equally.

Greedy regret minimization:

$$R_t = w_1 r_1 + w_2 r_2 + \cdots + w_t r_t$$

Idea

Choose how much influence each iteration should have.

Choosing the Weight

After computing a new regret vector:

$$R^{t+1} = R^t + wr^t$$

we choose:

$$w^* = \arg \min_w \Phi \left(\frac{R^t + wr^t}{W + w} \right)$$

$$\Phi(R) = \sum_i \max(R_i, 0)^2$$

Objective

Penalize remaining positive regret.

Implementation

Implemented an experimental framework in Julia.

Algorithms

- ▶ Regret Matching (RM)
- ▶ Regret Matching⁺ (RM⁺)
- ▶ Predictive RM (PRM)
- ▶ Greedy RM (GRM)
- ▶ Multiplicative Weights Update (MWU)
- ▶ Fictitious Play (FP)
- ▶ Double Oracle (DO) (external baseline)

Evaluation pipeline

- ▶ Benchmark game classes
- ▶ Algorithm comparison framework
- ▶ Greedy weight selection
- ▶ Convergence analysis
- ▶ Exploitability measurements

Experimental setup

Goal: compare convergence of regret minimization algorithms.

Game classes

- ▶ Random games
- ▶ Symmetric games
- ▶ Transitive games
- ▶ Non-transitive – RPS
- ▶ Structured games
 - ▶ Morra
 - ▶ Colonel Blotto
 - ▶ Matching Pennies

Evaluation

- ▶ Run algorithms iteratively
- ▶ Track average strategy
- ▶ Measure exploitability
- ▶ Compare convergence speed among algorithms

Exploitability

Evaluation metric

Measures how much a player can gain by switching to the best possible strategy.

$$\epsilon(\sigma) = 0 \iff \sigma \text{ is a Nash equilibrium}$$

Lower exploitability $\rightarrow$ better strategy

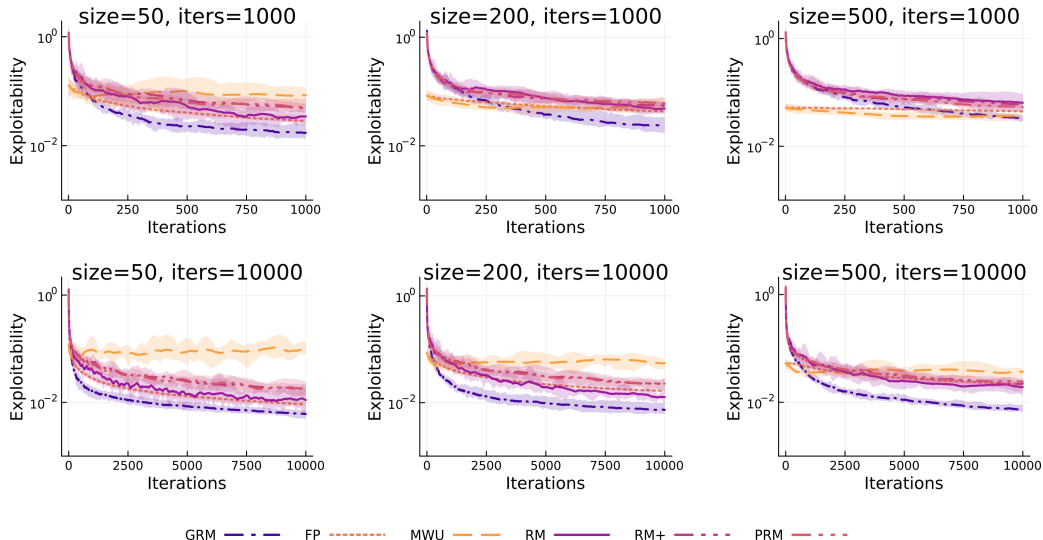
Results Overview

Game class	GRM performance
Matching Pennies	strongest convergence
Morra	consistently lowest exploitability
Blotto	competitive, improves with longer runs
RPS	stable, comparable to RM variants
Symmetric random	strong on medium/large games
Transitive	weak performance

Main observation

Greedy weighting works especially well in cyclic environments, but struggles with strict dominance structures.

Example Experiment: Symmetric Random Games



Comparison with Equilibrium Solvers

Until now: comparison with other learning dynamics.

Additional question:

How does GRM compare to algorithms that explicitly compute equilibria?

Double Oracle

- ▶ Builds a restricted game iteratively
- ▶ Computes equilibrium of each restricted game
- ▶ Adds new best-response strategies

Observed trade-off

- ▶ GRM: faster initial progress
- ▶ DO: lower exploitability with enough time

Questions & Discussion

Thank you for your attention.

Source code available on <https://github.com/SomiLotr/greedy-weights>